Merge Sort Performance on Partially Sorted Datasets

1. Aim

To evaluate the performance of **Merge Sort** on partially sorted datasets with different levels of disorder, record the execution time, and analyze how input characteristics affect the merging process.

2. Problem Statement

Merge Sort is a comparison-based sorting algorithm with a time complexity of **O(n log n)**. This experiment evaluates whether the amount of disorder in the input dataset affects Merge Sort's execution time. Different datasets are created with increasing levels of disorder and their execution times are compared.

3. Objectives

- To implement Merge Sort.
- To generate datasets with different levels of disorder.
- To measure the execution time for each dataset.
- To compare Merge Sort performance.
- To study how input characteristics affect merging.
- To understand the stability advantage of Merge Sort.

4. Input Characteristics

The experiment uses datasets with different degrees of disorder:

Dataset	Description
0% Disorder	Completely sorted
25% Disorder	Slightly shuffled

50% Disorder	Moderately shuffled
75% Disorder	Highly shuffled
100% Disorder	Completely random

The **number of elements is kept the same** for all datasets so that the comparison is fair.

5. Algorithm Used

Merge Sort

Merge Sort follows the **Divide and Conquer** technique.

1. Divide the array into two halves.
2. Recursively divide each half.
3. Sort the smaller subarrays.
4. Merge the sorted subarrays.
5. Continue until the complete array is sorted.

6. Algorithm

1. Start.
2. Generate a sorted dataset.
3. Introduce different amounts of disorder into copies of the dataset.
4. Apply Merge Sort to each dataset.
5. Record the execution time using a high-resolution timer.
6. Repeat the experiment to obtain reliable measurements.
7. Calculate the average execution time.
8. Compare the results.
9. Analyze the effect of disorder on merging.
10. Stop.

\

PROGRAM:

```python
import random
import time

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)


def merge(left, right):
    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
```

OUTPUT:

```
Merge Sort Performance
----------------------
0% Disorder : 0.017583 seconds
25% Disorder : 0.022263 seconds
50% Disorder : 0.024692 seconds
75% Disorder : 0.022794 seconds
100% Disorder : 0.024251 seconds
```

9. Performance Comparison

Degree of Disorder	Expected Performance
0%	Very similar

25%	Very similar
50%	Very similar
75%	Very similar
100%	Very similar

The important observation is that **Merge Sort's performance does not change dramatically with the degree of disorder**.

10. Analysis

Merge Sort divides the input into smaller subarrays regardless of whether the input is sorted or unsorted. Therefore, even a completely sorted dataset still undergoes the divide-and-merge process.

Partially sorted data can make some comparisons during merging easier because elements may already appear in the required order. However, Merge Sort still performs essentially the same recursive divisions and merging operations.

Therefore, the degree of disorder has **much less influence on Merge Sort** than it would on algorithms whose performance depends strongly on input ordering.

11. Stability Advantage

Merge Sort is a **stable sorting algorithm** when implemented using the `<=` comparison as shown in the code.

Stability means that if two elements have equal keys, their original relative order is preserved.

For example:

Before sorting:
(A, 80), (B, 80), (C, 70)

After stable sorting:
(C, 70), (A, 80), (B, 80)

`A` remains before `B` because both have the same key.

This is useful when sorting **student records, employee records, transaction data, or other datasets containing duplicate keys**.

12. Computational Complexity

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)

Space Complexity: `O(n)`

The important advantage is that Merge Sort maintains **O(n log n)** time complexity regardless of whether the input is sorted, partially sorted, or randomly ordered.

13. Result

The experiment shows that Merge Sort maintains relatively consistent execution time for datasets with different levels of disorder. Increasing disorder does not significantly change its asymptotic performance because the algorithm always follows the divide-and-merge process.

14. Conclusion

Merge Sort provides **predictable and stable performance** on partially sorted datasets. The amount of disorder has only a limited effect on its execution time. Its **O(n log n)** time complexity and stability make it suitable for large datasets and applications where preserving the order of equal elements is important.